CSE 213 — Software Engineering

Behavioral Design Patterns — Strategy, Template Method, Observer, Mediator, Command, State

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — behavioral-pattern questions by year
Lesson 5 — Strategy — pp. 4–48; SimUDuck, robots, sorting, talk-time, grading
Lesson 6 — Template Method — pp. 49–81; caffeine beverages, data miner, Template + Strategy
Lesson 7 — Observer — pp. 82–128; Weather-O-Rama, stock grabber, blogs, dashboards
Lesson 8 — Mediator — pp. 129–167; dialog, ATC tower, sort broker
Lesson 9 — Command — pp. 168–202; editor commands, undo, queueing, smart-home hub
Lesson 10 — State (and State vs Strategy) — pp. 203–260; gumball machine, document
Lesson 11 — Identifying the Pattern; Chain of Responsibility and Interpreter — mixed TF questions
Quick Notes — Behavioral Patterns — revision + key structures

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Page numbers refer to '2 - BehavioralDesignPattern.pdf' (261 pages, with animation build-ups). Chain of Responsibility and Interpreter are not in the slides; Lesson 11 gives the textbook version because 2023-24 Q2(b) and 2012-13 Q1(c) asked them. All Java compiled with javac (sources in _build/java/l2).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q5(b) Smart Home Hub: (i) Mediator vs direct links (ii) record/queue/replay actions — pattern (10) [L9] · Q8(a) Redesign DataParser with Template Method + Strategy (Java) (15) [L6] · Q8(b) Breaking-news displays update automatically — pattern (10) [L7] · Q8(c) State vs Strategy pattern (10) [L10]
2023-24 (CSE 213)	Q2(a) StockMarket / StockGrabber: UML with pattern + code for StockGrabber and StockObserver (22) [L7] · Q2(b) HelpHandler / Widget / Button / Dialog / Application: UML using the pattern (help requests) (8) [L11]
2022-23 (CSE 307)	Q2(a) Air-traffic control tower — class diagram + code (15) [L8] · Q4(a) BUET alumni blog notifies subscribers — class diagram + code (15) [L7]
2021-22 (CSE 307)	Q4(a) Identify patterns: (i) NuetFlix genre recommendations (ii) jit kommit/pussh/stashh (iii) BanglaPhone switch implementations (iv) R2-D2 anonymous reviews (20) [L11]
2020-21 (CSE 307)	Q2(a) Talk-time schemes added often: decouple UI programmers from scheme classes (10) [L5] · Q2(b) Strategy pattern for selection / insertion / quick sort (10) [L5] · Q2(c) Student IDs + marks → grades and merit list of top grade, using a pattern + code (15) [L5] · Q3(b) Dashboard of max/avg temperature updated at end of day (10) [L7]
2019-20 (CSE 307)	Q1(b) Central broker forwards sorting requests to three servers, with a queue — Mediator (20) [L8]
2018-19 (CSE 307)	Q2(a) Robots with Aggressive/Defensive/Normal behaviour changed at run time — class diagram + code (20) [L5]
2017-18 (CSE 307)	Q8(b) Blog notifies registered users of new articles — class diagram + code (17) [L7]
2012-13 (CSE 307)	Q1(c) Evaluate 'A op1 B op2 C': choose a pattern and draw the class diagram (14) [L11]
2010-11 (CSE 307)	Q4(c) Facebook group: join/leave, receive wall updates — pattern, class diagram, code (12) [L7]

EXAM FOCUS The biggest topic of IJ Mam's part. Observer is asked in 2024-25, 2023-24 (22 marks), 2022-23, 2020-21, 2017-18 and 2010-11; Strategy in 2020-21 (three parts) and 2018-19; Mediator in 2022-23 and 2019-20; Command + Mediator and Template + Strategy and State vs Strategy in 2024-25. Learn the six structures, then the comparison tables (Strategy vs Template, Mediator vs Observer, Command vs Mediator, State vs Strategy) — they are asked directly.

Lesson 5 — Strategy

Pages 4–48 (2 - BehavioralDesignPattern)

Behavioral patterns deal with how objects behave and interact: communication, flow of responsibility and how tasks are carried out between objects (pp. 2–3).

1. SimUDuck (pp. 4–39)

- Duck superclass with quack(), swim(), display(), and later fly() → RubberDuck inherits fly() and flies (p. 8).
- Override fly() to do nothing in RubberDuck, DecoyDuck … → every new duck must be checked (pp. 9–12).
- Flyable / Quackable interfaces → duplicated fly code in every class; a change touches all (pp. 13–18).
- Principle (p. 19): identify the aspects that vary and separate them from what stays the same.
- Duck **has** FlyBehavior and QuackBehavior (FlyWithWings, FlyNoWay; Quack, Squeak, MuteQuack); performFly() delegates; setFlyBehavior() changes it at run time — ModelDuck gets FlyRocketPowered (pp. 20–39).

> MEMORIZE **Strategy** — Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it (p. 45).

```
Client --creates strategy--> Context - strategy: Strategy ---------> «interface» Strategy + execute(data)
                             + setStrategy(s)                              ^            ^
                             + doSomething() { strategy.execute(data); }  ConcreteStrategyA  ConcreteStrategyB
```

Navigator example (pp. 42–44): routeStrategy = Road / PublicTransport / Walking, each buildRoute(A, B). Payment (p. 47): Credit card vs PayPal.

Use Strategy when	Not when
several variants of one algorithm/behaviour; choose or switch at run time	the variants are steps of one fixed skeleton → Template Method
a class has a big conditional choosing the behaviour	behaviour depends on the object's own state and changes itself → State

Practice — Lesson 5

(a) Actual — 2018-19 Q2(a). (b) Actual — 2020-21 Q2(b). (c) Actual — 2020-21 Q2(a). (d) Actual — 2020-21 Q2(c).

(a) Robots: interface IBehaviour, AggressiveBehaviour, DefensiveBehaviour, NormalBehaviour; Robot passes sensor data (position, close obstacles) to its behaviour; Big Robot is aggressive, George v.2.1 defensive, R2 normal; later the behaviours change. Class diagram + code. (20)

(b) Three sorting routines (selection, insertion, quicksort): demonstrate Strategy. (10)

(c) A web system shows mobile talk-time schemes and their cost; new schemes are added often. Decouple the UI programmers from the scheme classes. (10)

(d) Student IDs with marks: find grades and prepare the merit list of the students with the highest grade, using a pattern; any grading scheme. (15)

✓ Solution

(a) Strategy: Robot = context, Behaviour = strategy interface, three concrete behaviours; setBehaviour() switches at run time.

```
Robot - name, position, behaviour: Behaviour --------> «interface» Behaviour + moveCommand(name, pos,
obstacle): int
  + setBehaviour(b)  + move(obstacleClose)                  ^ AggressiveBehaviour ^
DefensiveBehaviour ^ NormalBehaviour
```

```
// 2018-19 Q2(a) — Strategy: robot behaviours changed at run time
interface Behaviour { int moveCommand(String robot, int position, boolean obstacleClose); }
class AggressiveBehaviour implements Behaviour {
    public int moveCommand(String r, int pos, boolean obs) {
        System.out.println(r + ": aggressive - attack the robot found"); return obs ? 1 : 0; }
}
class DefensiveBehaviour implements Behaviour {
    public int moveCommand(String r, int pos, boolean obs) {
        System.out.println(r + ": defensive - run away in the opposite direction"); return obs ? -1 :
0; }
}
class NormalBehaviour implements Behaviour {
    public int moveCommand(String r, int pos, boolean obs) {
        System.out.println(r + ": normal - ignore other robots"); return 0; }
}
class Robot {                                          // context
    private final String name; private Behaviour behaviour; private int position;
    Robot(String name) { this.name = name; }
    void setBehaviour(Behaviour b) { behaviour = b; }
    void move(boolean obstacleClose) {                 // passes sensor info to the strategy
        position += behaviour.moveCommand(name, position, obstacleClose);
    }
}
public class RobotSim {
    public static void main(String[] args) {
        Robot r1 = new Robot("Big Robot"), r2 = new Robot("George v.2.1"), r3 = new Robot("R2");
        r1.setBehaviour(new AggressiveBehaviour());
        r2.setBehaviour(new DefensiveBehaviour());
        r3.setBehaviour(new NormalBehaviour());
        r1.move(true); r2.move(true); r3.move(true);
        System.out.println("-- behaviours changed --");
        r1.setBehaviour(new DefensiveBehaviour());
        r2.setBehaviour(new NormalBehaviour());
        r3.setBehaviour(new AggressiveBehaviour());
        r1.move(true); r2.move(true); r3.move(true);
    }
}
```

(b) Sorter (context) holds a SortStrategy; three concrete strategies.

```
// 2020-21 Q2(b) – Strategy: selection / insertion / quick sort
import java.util.Arrays;

interface SortStrategy { void sort(int[] a); }
class SelectionSort implements SortStrategy {
    public void sort(int[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            int m = i;
            for (int j = i + 1; j < a.length; j++) if (a[j] < a[m]) m = j;
            int t = a[i]; a[i] = a[m]; a[m] = t;
        }
    }
}
class InsertionSort implements SortStrategy {
    public void sort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int k = a[i], j = i - 1;
            while (j >= 0 && a[j] > k) { a[j + 1] = a[j]; j--; }
            a[j + 1] = k;
        }
    }
}
class QuickSort implements SortStrategy {
    public void sort(int[] a) { qs(a, 0, a.length - 1); }
    private void qs(int[] a, int lo, int hi) {
        if (lo >= hi) return;
        int p = a[hi], i = lo;
        for (int j = lo; j < hi; j++) if (a[j] < p) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; }
        int t = a[i]; a[i] = a[hi]; a[hi] = t;
        qs(a, lo, i - 1); qs(a, i + 1, hi);
    }
}
class Sorter {                                          // context
    private SortStrategy strategy;
    Sorter(SortStrategy s) { strategy = s; }
    void setStrategy(SortStrategy s) { strategy = s; }
    int[] sort(int[] data) { int[] c = data.clone(); strategy.sort(c); return c; }
}
```

```
public class SortStrategyDemo {
    public static void main(String[] args) {
        int[] d = {5, 2, 9, 1, 7, 3};
        Sorter s = new Sorter(new SelectionSort());
        System.out.println("selection " + Arrays.toString(s.sort(d)));
        s.setStrategy(new InsertionSort());
        System.out.println("insertion " + Arrays.toString(s.sort(d)));
        s.setStrategy(new QuickSort());
        System.out.println("quick     " + Arrays.toString(s.sort(d)));
    }
}
```

(c) Strategy: every scheme implements TalkTimeScheme (name(), cost(minutes)); the UI page depends only on that interface and receives the list from a registry/factory. Adding a scheme = one new class + one registry line; the UI code never changes.

```
// 2020-21 Q2(a) — Strategy: UI depends only on the TalkTimeScheme interface; new schemes are plug-ins
import java.util.ArrayList;
import java.util.List;

interface TalkTimeScheme { String name(); double cost(int minutes); }
class PerMinuteScheme implements TalkTimeScheme {
    public String name() { return "Per minute (Tk 1.0/min)"; }
    public double cost(int m) { return m * 1.0; }
}
class BundleScheme implements TalkTimeScheme {
    public String name() { return "Bundle 300 min for Tk 199, then Tk 0.8/min"; }
    public double cost(int m) { return 199 + Math.max(0, m - 300) * 0.8; }
}
class NightScheme implements TalkTimeScheme {          // added later: no UI change
    public String name() { return "Night owl (Tk 0.5/min)"; }
    public double cost(int m) { return m * 0.5; }
}
class SchemeRegistry {                                  // the only place that knows concrete schemes
    static List<TalkTimeScheme> all() {
        List<TalkTimeScheme> l = new ArrayList<>();
        l.add(new PerMinuteScheme()); l.add(new BundleScheme()); l.add(new NightScheme());
        return l;
    }
}
class SchemePage {                                      // UI team codes against the interface only
    void show(TalkTimeScheme s, int minutes) {
        System.out.printf("%-45s %4d min -> Tk %.2f%n", s.name(), minutes, s.cost(minutes));
    }
}
public class TalkTimeDemo {
    public static void main(String[] args) {
        SchemePage ui = new SchemePage();
        for (TalkTimeScheme s : SchemeRegistry.all()) ui.show(s, 400);
    }
}
```

(d) Strategy for the grading scheme (BUET letter grades or pass/fail chosen at run time); ResultProcessor computes grades and the merit list (students with the top grade, sorted by marks).

```
// 2020-21 Q2(c) — Strategy: grading scheme chosen at run time; merit list of the top grade
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

interface GradingStrategy { String grade(int marks); String topGrade(); }
class BuetGrading implements GradingStrategy {
    public String grade(int m) {
        if (m >= 80) return "A+"; if (m >= 75) return "A"; if (m >= 70) return "A-";
        if (m >= 65) return "B+"; if (m >= 60) return "B"; if (m >= 55) return "B-";
        if (m >= 50) return "C+"; if (m >= 45) return "C"; if (m >= 40) return "D"; return "F";
    }
    public String topGrade() { return "A+"; }
}
class PassFailGrading implements GradingStrategy {
    public String grade(int m) { return m >= 50 ? "P" : "F"; }
    public String topGrade() { return "P"; }
}
class ResultProcessor {                                  // context
    private GradingStrategy strategy;
    ResultProcessor(GradingStrategy s) { strategy = s; }
    void setStrategy(GradingStrategy s) { strategy = s; }
    Map<String, String> grades(Map<String, Integer> marks) {
        Map<String, String> g = new LinkedHashMap<>();
        for (Map.Entry<String, Integer> e : marks.entrySet()) g.put(e.getKey(),
strategy.grade(e.getValue()));
        return g;
    }
    List<String> meritList(Map<String, Integer> marks) {   // students with the highest grade, by
marks
        List<String> ids = new ArrayList<>();
        for (Map.Entry<String, Integer> e : marks.entrySet())
            if (strategy.grade(e.getValue()).equals(strategy.topGrade())) ids.add(e.getKey());
        ids.sort((a, b) -> marks.get(b) - marks.get(a));
        return ids;
    }
}
public class MeritListDemo {
    public static void main(String[] args) {
        Map<String, Integer> marks = new LinkedHashMap<>();
        marks.put("2205001", 86); marks.put("2205002", 74); marks.put("2205003", 91);
marks.put("2205004", 38);
        ResultProcessor rp = new ResultProcessor(new BuetGrading());
        System.out.println(rp.grades(marks));
        System.out.println("Merit list: " + rp.meritList(marks));
    }
}
```

Lesson 6 — Template Method

Pages 49–81 (2 - BehavioralDesignPattern) · Misc pp. 3–4

Coffee: boil water, brew coffee grinds, pour in cup, add sugar and milk. Tea: boil water, steep tea bag, pour in cup, add lemon. Same skeleton (pp. 49–63) → CaffeineBeverage.prepareRecipe() is **final**: boilWater(); brew(); pourInCup(); addCondiments(); — brew() and addCondiments() are abstract; Coffee and Tea implement them.

Hook (p. 72): if (customerWantsCondiments()) addCondiments(); — customerWantsCondiments() returns true by default and subclasses may override it.

> **MEMORIZE** **Template Method** defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure (p. 73).

```
AbstractClass
 + templateMethod() {final}  { step1(); step2(); if (hook()) step3(); step4(); }
 # step1(), step4()          (implemented: common)
 # step2(), step3()          {abstract}
 # hook(): boolean           (default implementation, may be overridden)
        ^                          ^
 ConcreteClass1             ConcreteClass2   (override step2, step3, optionally hook)
```

Data mining (pp. 75–77): DocDataMiner, CSVDataMiner, PDFDataMiner repeat openFile, extractData, parseData, analyzeData, sendReport, closeFile → DataMiner.mine(path) template; format-specific steps abstract; analyzeData and sendReport shared.

	Template Method	Strategy
Based on	inheritance — alter parts by extending them in subclasses	composition — alter behaviour by supplying a different strategy object
Level / time	class level, static (fixed at compile time)	object level, can switch at run time
Controls	superclass controls the order of steps	context delegates the whole algorithm

Practice — Lesson 6

(a) Actual — 2024-25 Q8(a).

(a) A DataParser reads data files in different formats (CSV, JSON …) through the same steps (open, extract, parse, analyse, report, close) but the analysis algorithm must be selectable. Redesign it using Template Method and Strategy together, in Java. (15)

✓ Solution

(a) Template Method fixes the pipeline in a final process() method; format-specific steps (extractData, parseData) are abstract and implemented by CsvParser and JsonParser; wantsReport() is a hook. The analysis step varies independently of the format, so it is a Strategy object injected into the parser — any format can be combined with any analysis without new subclasses.

```
DataParser {abstract} - analysis: AnalysisStrategy -------> «interface» AnalysisStrategy +
analyze(values)
 + process(raw) {final}  (template)                              ^ AverageAnalysis   ^ MaxAnalysis
 # openFile(), extractData(){abstract}, parseData(){abstract}, wantsReport() (hook), sendReport(),
closeFile()
     ^ CsvParser    ^ JsonParser
```

```
// 2024-25 Q8(a) — Template Method (fixed parsing pipeline) + Strategy (pluggable analysis)
import java.util.Arrays;
import java.util.List;

interface AnalysisStrategy { String analyze(List<Double> values); }     // strategy
class AverageAnalysis implements AnalysisStrategy {
    public String analyze(List<Double> v) {
        return "average = " + v.stream().mapToDouble(Double::doubleValue).average().orElse(0); }
}
class MaxAnalysis implements AnalysisStrategy {
    public String analyze(List<Double> v) {
        return "max = " + v.stream().mapToDouble(Double::doubleValue).max().orElse(0); }
}
abstract class DataParser {                                              // template
    private final AnalysisStrategy analysis;
    DataParser(AnalysisStrategy a) { analysis = a; }
    public final String process(String raw) {                            // the template method
        String content = openFile(raw);
        List<Double> data = parseData(extractData(content));
        String result = analysis.analyze(data);                          // varying algorithm =
strategy
        if (wantsReport()) sendReport(result);                           // hook
        closeFile();
        return result;
    }
    protected String openFile(String raw) { return raw.trim(); }         // common steps
    protected abstract String extractData(String content);               // format-specific steps
    protected abstract List<Double> parseData(String data);
    protected boolean wantsReport() { return true; }                     // hook with default
    protected void sendReport(String r) { System.out.println("Report: " + r); }
    protected void closeFile() { }
}
class CsvParser extends DataParser {
    CsvParser(AnalysisStrategy a) { super(a); }
    protected String extractData(String c) { return c; }
    protected List<Double> parseData(String d) {
        return Arrays.stream(d.split(",")).map(String::trim).map(Double::valueOf).toList(); }
}
class JsonParser extends DataParser {
    JsonParser(AnalysisStrategy a) { super(a); }
    protected String extractData(String c) { return c.substring(c.indexOf('[') + 1, c.indexOf(']')); }
}
    protected List<Double> parseData(String d) {
        return Arrays.stream(d.split(",")).map(String::trim).map(Double::valueOf).toList(); }
    protected boolean wantsReport() { return false; }
}
```

```
public class DataMinerDemo {
    public static void main(String[] args) {
        System.out.println(new CsvParser(new AverageAnalysis()).process("3, 5, 10"));
        System.out.println(new JsonParser(new MaxAnalysis()).process("{\"v\": [4, 9, 2]}"));
    }
}
```

Lesson 7 — Observer

Pages 82–128 (2 - BehavioralDesignPattern)

Weather-O-Rama (pp. 82–94): WeatherData has getTemperature(), getHumidity(), getPressure() and measurementsChanged(); three displays (current conditions, statistics, forecast) must update, and more may be added. Coding the displays into measurementsChanged() couples WeatherData to concrete classes. Newspaper subscription (pp. 95–107): publisher + subscribers who can subscribe and unsubscribe.

MEMORIZE **Observer** — Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically (p. 121).

```
«interface» Subject                                    «interface» Observer
 + registerObserver(o)  + removeObserver(o)              + update(temp, humidity, pressure)
 + notifyObservers()                                       ^            ^              ^
        ^                                          CurrentConditions  StatisticsDisplay  ForecastDisplay
   WeatherData - observers: List<Observer>         (each has display(); its constructor calls
   + measurementsChanged() { notifyObservers(); }  weatherData.registerObserver(this))
```

General form (p. 121–127): Publisher (subscribers[], mainState; subscribe(s), unsubscribe(s), notifySubscribers() { for each s: s.update(this) }) and «interface» Subscriber update(context). **Push** = subject sends the data in update(); **pull** = subject sends itself and observers read what they need. Observer = event listener (p. 128).

Practice — Lesson 7

(a) Actual — 2023-24 Q2(a). (b) Actual — 2022-23 Q4(a) / 2017-18 Q8(b). (c) Actual — 2020-21 Q3(b). (d) Actual — 2024-25 Q8(b). (e) Actual — 2010-11 Q4(c).

(a) StockMarket: StockGrabber holds the prices; StockObserver objects must be updated when any price changes. (i) UML with the pattern (6) (ii) code StockGrabber and StockObserver (8×2).

(b) BUET alumni register to a blog; when a new article is posted the blog notifies them; they then open the blog and read it. Class diagram + code. (15 / 5+12)

(c) A dashboard shows the max and average temperature, updated at the end of each day. Pattern + design. (10)

(d) Breaking-news items must appear on several displays (TV ticker, mobile app, website) as soon as they arrive. (10)

(e) Facebook group: users join and leave; every wall post is broadcast to all members' news feeds. Pattern, class diagram and code fragments. (12)

✓ Solution

(a)

```
«interface» Subject + register(o), unregister(o), notifyObservers()      «interface» Observer + update(ibm,
aapl, goog)
        ^                                                                        ^
StockGrabber - observers: List<Observer>                                 StockObserver - id, stockGrabber:
Subject
  - ibmPrice, aaplPrice, googPrice                                        + update(...)  (prints the
prices)
  + setIBMPrice(p) / setAAPLPrice(p) / setGOOGPrice(p) -> notifyObservers()
StockObserver ----registers with----> StockGrabber (1 subject, 0..* observers)
```

```
// 2023-24 Q2(a) – Observer: StockGrabber (subject) and StockObserver (observer)
import java.util.ArrayList;
import java.util.List;

interface Subject {
    void register(Observer o);
    void unregister(Observer o);
    void notifyObservers();
}
interface Observer { void update(double ibmPrice, double aaplPrice, double googPrice); }

class StockGrabber implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    private double ibmPrice, aaplPrice, googPrice;
    public void register(Observer o)   { observers.add(o); }
    public void unregister(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) o.update(ibmPrice, aaplPrice, googPrice);
    }
    public void setIBMPrice(double p)  { ibmPrice = p;  notifyObservers(); }
    public void setAAPLPrice(double p) { aaplPrice = p; notifyObservers(); }
    public void setGOOGPrice(double p) { googPrice = p; notifyObservers(); }
}
class StockObserver implements Observer {
    private static int count = 0;
    private final int id;
    private final Subject stockGrabber;
    StockObserver(Subject s) {
        stockGrabber = s;
        id = ++count;
        s.register(this);                               // subscribes itself
        System.out.println("New observer " + id);
    }
    public void update(double ibm, double aapl, double goog) {
        System.out.printf("Observer %d: IBM %.2f AAPL %.2f GOOG %.2f%n", id, ibm, aapl, goog);
    }
    void stop() { stockGrabber.unregister(this); }
}
```

```
public class StockDemo {
    public static void main(String[] args) {
        StockGrabber g = new StockGrabber();
        StockObserver o1 = new StockObserver(g);
        g.setIBMPrice(197.00);
        StockObserver o2 = new StockObserver(g);
        g.setAAPLPrice(677.60);
        o1.stop();
        g.setGOOGPrice(676.40);                         // only observer 2 is told
    }
}
```

(b) Pull model: update(blog) tells the user that a new article is posted; the user then calls blog.getLatestArticle() — exactly 'then the user will access the blog and read'.

```
// 2022-23 Q4(a), 2017-18 Q8(b) — Observer: alumni blog notifies registered users; they pull the new
article
import java.util.ArrayList;
import java.util.List;

interface BlogObserver { void update(Blog blog); }
class Blog {                                              // subject (publisher)
    private final List<BlogObserver> subscribers = new ArrayList<>();
    private final List<String> articles = new ArrayList<>();
    void register(BlogObserver o)   { subscribers.add(o); }
    void unregister(BlogObserver o) { subscribers.remove(o); }
    void postArticle(String title) {
        articles.add(title);
        for (BlogObserver o : subscribers) o.update(this);  // 'a new article is posted'
    }
    String getLatestArticle() { return articles.get(articles.size() - 1); }
}
class AlumniUser implements BlogObserver {               // concrete observer
    private final String name;
    AlumniUser(String name) { this.name = name; }
    public void update(Blog blog) {
        System.out.println(name + " notified: a new article is posted");
        System.out.println(name + " reads: " + blog.getLatestArticle());   // pull model
    }
}
public class BlogDemo {
    public static void main(String[] args) {
        Blog blog = new Blog();
        AlumniUser a = new AlumniUser("Asef"), b = new AlumniUser("Nadia");
        blog.register(a); blog.register(b);
        blog.postArticle("Life after BUET");
        blog.unregister(a);
        blog.postArticle("Alumni reunion 2026");
    }
}
```

(c) Observer: TemperatureLog (subject) collects readings and at endOfDay() pushes (max, avg) to every attached widget; new widgets just attach.

```
// 2020-21 Q3(b) — Observer: end-of-day temperature summary pushed to dashboards
import java.util.ArrayList;
import java.util.List;

interface DayObserver { void update(double max, double avg); }
class TemperatureLog {                                    // subject
    private final List<Double> readings = new ArrayList<>();
    private final List<DayObserver> observers = new ArrayList<>();
    void attach(DayObserver o) { observers.add(o); }
    void record(double t) { readings.add(t); }
    void endOfDay() {
        double max = readings.stream().mapToDouble(Double::doubleValue).max().orElse(0);
        double avg = readings.stream().mapToDouble(Double::doubleValue).average().orElse(0);
        for (DayObserver o : observers) o.update(max, avg);
        readings.clear();
    }
}
class MaxTempWidget implements DayObserver {
    public void update(double max, double avg) { System.out.println("Max today: " + max); }
}
class AvgTempWidget implements DayObserver {
    public void update(double max, double avg) { System.out.printf("Average today: %.2f%n", avg); }
}
public class TempDashboard {
    public static void main(String[] args) {
        TemperatureLog log = new TemperatureLog();
        log.attach(new MaxTempWidget()); log.attach(new AvgTempWidget());
        for (double t : new double[]{24.5, 31.0, 29.5, 27.0}) log.record(t);
        log.endOfDay();
    }
}
```

(d) Observer: NewsAgency (subject) keeps a list of NewsDisplay observers (TvTicker, MobileApp, Website); publish(news) calls display.update(news) on each. Displays subscribe/unsubscribe at run time; the agency does not know their concrete classes. Same code shape as (a).

(e) Observer: Group = subject with join(user) = register, leave(user) = remove, postOnWall(msg) → notifyMembers() → user.update(group, msg) adds it to the user's news feed. Group also has title, admins, type (open/closed) and related groups. For groups inside groups see the Composite extension in the Structural notes (2010-11 Q4(d)).

Lesson 8 — Mediator

Pages 129–167 (2 - BehavioralDesignPattern)

Air-traffic control (p. 129): pilots do not talk to each other; all communication goes through the tower. Customer profile dialog (pp. 131–150): checkboxes, text fields and buttons that react to each other create a web of dependencies; instead the Dialog is the mediator — components only notify it.

> **MEMORIZE** **Mediator** restricts direct communications between objects and forces them to collaborate only via a mediator object; it reduces chaotic dependencies (p. 157).

```
ComponentA - m: Mediator   ComponentB - m: Mediator  ...   (each calls m.notify(this, event))
        \                        |                        /
         +--------> «interface» Mediator + notify(sender, event) <-------+
                                 ^
                  ConcreteMediator - componentA, componentB, ...
                  + notify(sender, event) { if (sender == componentA) reactOnA(); ... }
```

	Mediator	Observer
Goal	eliminate mutual dependencies among a set of components; all depend on one mediator	establish dynamic one-way connections: subscribers follow a publisher
Direction	many ↔ one ↔ many (two-way coordination logic in the mediator)	one → many notifications
Note	a mediator is often implemented with observer-style notify()	observers can come and go at run time

Practice — Lesson 8

(a) Actual — 2022-23 Q2(a). (b) Actual — 2019-20 Q1(b).

(a) ATC coordinates landing and take-off; flights must not depend on each other. Class diagram + code. (15)

(b) Three servers sort with selection, insertion and quicksort; a central broker receives (algorithm, data), forwards to the right server, returns the sorted array, and queues requests that arrive together. Demonstrate Mediator. (20)

✓ Solution

```
«interface» ATCMediator + register(f), requestLanding(f): boolean, landed(f)
        ^ ControlTower - flights: List<Flight> - runwayFree: boolean
Flight - id - atc: ATCMediator + land() + receive(msg)       Flight 0..* ---- 1 ControlTower
```

```
// 2022-23 Q2(a) — Mediator: flights talk only to the ATC tower
import java.util.ArrayList;
import java.util.List;

interface ATCMediator {
    void register(Flight f);
    boolean requestLanding(Flight f);
    void landed(Flight f);
}
class ControlTower implements ATCMediator {
    private final List<Flight> flights = new ArrayList<>();
    private boolean runwayFree = true;
    public void register(Flight f) { flights.add(f); }
    public boolean requestLanding(Flight f) {
        if (runwayFree) {
            runwayFree = false;
            for (Flight o : flights) if (o != f) o.receive(f.id + " is landing, hold your position");
            return true;
        }
        return false;
    }
    public void landed(Flight f) {
        runwayFree = true;
        for (Flight o : flights) if (o != f) o.receive("runway free after " + f.id);
    }
}
class Flight {                                          // colleague: knows only the mediator
    final String id; private final ATCMediator atc;
    Flight(String id, ATCMediator atc) { this.id = id; this.atc = atc; atc.register(this); }
    void land() {
        if (atc.requestLanding(this)) { System.out.println(id + ": landing"); atc.landed(this); }
        else System.out.println(id + ": waiting - runway busy");
    }
    void receive(String msg) { System.out.println(id + " <- ATC: " + msg); }
}
public class AtcDemo {
    public static void main(String[] args) {
        ATCMediator tower = new ControlTower();
        Flight bg = new Flight("BG-147", tower), ek = new Flight("EK-585", tower);
        bg.land();
        ek.land();
    }
}
```

(b) CentralBroker is the concrete mediator; clients and servers are colleagues that never reference each other. A FIFO queue holds simultaneous requests.

```
Client - broker: Broker + requestSort(alg, data) + receive(sorted)
«interface» Broker + submit(client, alg, data) + processAll()
   ^ CentralBroker - servers: Map<String, SortServer> - queue: Queue<Request>
SortServer {abstract} + algorithm() + sort(a)   ^ SelectionServer ^ InsertionServer ^ QuickServer
```

```
// 2019-20 Q1(b) — Mediator: central broker forwards sorting requests to servers, queueing them
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.Queue;

interface Broker { void submit(Client c, String algorithm, int[] data); void processAll(); }

abstract class SortServer {                                  // colleague
    abstract String algorithm();
    abstract int[] sort(int[] a);
}
class SelectionServer extends SortServer {
    String algorithm() { return "selection"; }
    int[] sort(int[] a) {
        int[] c = a.clone();
        for (int i = 0; i < c.length - 1; i++) { int m = i;
            for (int j = i + 1; j < c.length; j++) if (c[j] < c[m]) m = j;
            int t = c[i]; c[i] = c[m]; c[m] = t; }
        return c;
    }
}
class InsertionServer extends SortServer {
    String algorithm() { return "insertion"; }
    int[] sort(int[] a) {
        int[] c = a.clone();
        for (int i = 1; i < c.length; i++) { int k = c[i], j = i - 1;
            while (j >= 0 && c[j] > k) { c[j + 1] = c[j]; j--; } c[j + 1] = k; }
        return c;
    }
}
class QuickServer extends SortServer {
    String algorithm() { return "quick"; }
    int[] sort(int[] a) { int[] c = a.clone(); Arrays.sort(c); return c; }   // library quicksort for
primitives
}
class Client {                                               // colleague
    private final String name; private final Broker broker;
    Client(String name, Broker b) { this.name = name; broker = b; }
    void requestSort(String alg, int[] data) { broker.submit(this, alg, data); }
    void receive(int[] sorted) { System.out.println(name + " got " + Arrays.toString(sorted)); }
}
```

```
class CentralBroker implements Broker {                      // concrete mediator
    private record Request(Client c, String alg, int[] data) { }
    private final Map<String, SortServer> servers = new HashMap<>();
    private final Queue<Request> queue = new ArrayDeque<>();
    void addServer(SortServer s) { servers.put(s.algorithm(), s); }
    public void submit(Client c, String alg, int[] data) { queue.add(new Request(c, alg, data)); }
    public void processAll() {                               // FIFO when several requests arrive
together
        while (!queue.isEmpty()) {
            Request r = queue.poll();
            r.c().receive(servers.get(r.alg()).sort(r.data()));
        }
    }
}
public class SortBroker {
    public static void main(String[] args) {
        CentralBroker b = new CentralBroker();
        b.addServer(new SelectionServer()); b.addServer(new InsertionServer()); b.addServer(new
QuickServer());
        Client c1 = new Client("C1", b), c2 = new Client("C2", b);
        c1.requestSort("quick", new int[]{4, 1, 3});
        c2.requestSort("selection", new int[]{9, 7, 8});
        c1.requestSort("insertion", new int[]{2, 0, 1});
        b.processAll();
    }
}
```

Lesson 9 — Command

Pages 168–202 (2 - BehavioralDesignPattern)

Text editor (pp. 168–188): a Button subclass per action (SaveButton, CopyButton …) explodes and mixes GUI with business logic; the same action is triggered from a button, a menu and Ctrl+S. Put a **command** object between the GUI layer and the business logic: all commands implement execute(); SaveCommand, OpenCommand, PrintCommand.

Undo (pp. 189–191): Command keeps app, editor and a backup (saveBackup(), undo(), execute()); CopyCommand, CutCommand, PasteCommand, UndoCommand; CommandHistory is a stack (push, pop); Application.executeCommand(c) pushes commands that changed state; undo() pops and calls c.undo().

MEMORIZE **Command** turns a request into a stand-alone object that contains all information about the request. This lets you parameterize methods with different requests, delay or queue a request's execution, and support undoable operations (p. 192).

```
Client --creates--> ConcreteCommand - receiver - params      Invoker - command: Command
                    + execute() { receiver.operation(params); }   + setCommand(c) + executeCommand()
{ command.execute(); }
                         ^ implements                          Receiver + operation(a, b, c)
                    «interface» Command + execute()
```

	Command	Mediator
Connection	unidirectional sender → receiver through a command object	removes direct connections; colleagues talk via the mediator
Extra	queue, log, schedule, undo/redo, macros (p. 201)	central coordination logic

Practice — Lesson 9

(a) Actual — 2024-25 Q5(b). (b) Actual — 2021-22 Q4(a)(ii).

(a) A Smart Home Hub coordinates motion sensors, lights, alarms and other devices. (i) A motion event may trigger one or more coordinated actions (light on, alarm, notify the hub): show why Mediator is preferable to letting the motion sensor talk to the devices directly. (ii) The system must record, queue and later replay sensor-triggered actions: choose a behavioral pattern and show how. (10)

(b) Version-control system jit: kommit (save locally), pussh (sync kommits with remote), stashh (discard un-pushed kommits, get latest remote). Pattern?

✓ Solution

(a)(i) Direct links: MotionSensor would hold references to Light, Alarm, Notifier and contain the rules → N sensors × M devices dependencies; adding a device or changing a rule edits every sensor. With **Mediator** the sensor only calls hub.notify(this, "motion"); the hub owns the coordination rules, devices and sensors stay independent and reusable, and rules change in one place.

(ii) Command: each action (LightOnCommand, AlarmCommand, NotifyCommand) is an object with execute(); the hub (invoker) puts commands in a queue, executes them, and appends them to a history list that can be replayed (or undone if commands store state).

```
// 2024-25 Q5(b) — Mediator (hub) + Command (record, queue, replay sensor-triggered actions)
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;

interface Command { void execute(); String name(); }        // command
class Light { void on() { System.out.println("Light ON"); } }
class Alarm { void activate() { System.out.println("Alarm ACTIVATED"); } }
class Notifier { void send(String m) { System.out.println("Notification: " + m); } }
class LightOnCommand implements Command {
    private final Light light; LightOnCommand(Light l) { light = l; }
    public void execute() { light.on(); } public String name() { return "LightOn"; }
}
class AlarmCommand implements Command {
    private final Alarm alarm; AlarmCommand(Alarm a) { alarm = a; }
    public void execute() { alarm.activate(); } public String name() { return "Alarm"; }
}
class NotifyCommand implements Command {
    private final Notifier n; private final String msg;
    NotifyCommand(Notifier n, String msg) { this.n = n; this.msg = msg; }
    public void execute() { n.send(msg); } public String name() { return "Notify"; }
}
class SmartHomeHub {                                         // mediator + invoker
    private final Queue<Command> pending = new ArrayDeque<>();
    private final List<Command> history = new ArrayList<>();
    private final Light light = new Light(); private final Alarm alarm = new Alarm(); private final
Notifier n = new Notifier();
    void notify(String sender, String event) {               // sensors call only the hub
        if (sender.equals("MotionSensor") && event.equals("motion")) {
            pending.add(new LightOnCommand(light));
            pending.add(new NotifyCommand(n, "motion in hallway"));
        } else if (sender.equals("DoorSensor") && event.equals("forced")) {
            pending.add(new AlarmCommand(alarm));
        }
    }
    void runQueued() { while (!pending.isEmpty()) { Command c = pending.poll(); c.execute();
history.add(c); } }
    void replay() { System.out.println("-- replay --"); for (Command c : history) c.execute(); }
}
class MotionSensor {                                         // colleague
    private final SmartHomeHub hub; MotionSensor(SmartHomeHub h) { hub = h; }
    void detect() { hub.notify("MotionSensor", "motion"); }
}
```

```
public class SmartHomeDemo {
    public static void main(String[] args) {
        SmartHomeHub hub = new SmartHomeHub();
        new MotionSensor(hub).detect();
        hub.notify("DoorSensor", "forced");
        hub.runQueued();
        hub.replay();
    }
}
```

(b) Command: Kommit, Pussh, Stashh are command objects with execute() acting on the Repo receiver; the jit invoker logs them (history, possible undo).

```
// 2021-22 Q4(a)(ii) — Command: kommit / pussh / stashh as command objects with a history
import java.util.ArrayList;
import java.util.List;

class Repo {                                                    // receiver
    final List<String> local = new ArrayList<>();      // kommitted, not pushed
    final List<String> remote = new ArrayList<>();
    void commit(String c) { local.add(c); }
    void push() { remote.addAll(local); local.clear(); }
    void stash() { local.clear(); }                    // discard un-pushed commits, keep remote
}
interface VcsCommand { void execute(); }
class Kommit implements VcsCommand { private final Repo r; private final String msg;
    Kommit(Repo r, String m) { this.r = r; msg = m; } public void execute() { r.commit(msg); } }
class Pussh implements VcsCommand { private final Repo r; Pussh(Repo r) { this.r = r; } public void
execute() { r.push(); } }
class Stashh implements VcsCommand { private final Repo r; Stashh(Repo r) { this.r = r; } public void
execute() { r.stash(); } }
class Jit {                                                     // invoker
    private final List<VcsCommand> log = new ArrayList<>();
    void run(VcsCommand c) { c.execute(); log.add(c); }
}
public class VcsDemo {
    public static void main(String[] args) {
        Repo repo = new Repo(); Jit jit = new Jit();
        jit.run(new Kommit(repo, "a")); jit.run(new Pussh(repo));
        jit.run(new Kommit(repo, "b")); jit.run(new Stashh(repo));
        System.out.println("local " + repo.local + " remote " + repo.remote);
    }
}
```

Lesson 10 — State (and State vs Strategy)

Pages 203–260 (2 - BehavioralDesignPattern)

Mighty Gumball (pp. 203–216): states No Quarter, Has Quarter, Gumball Sold, Out of Gumballs; actions insertQuarter, ejectQuarter, turnCrank, dispense. Coding it with int constants and an if-else on the state in every action works, but the new 'winner' rule (10 % of the time two gumballs) forces changes in every method — the nightmare of change.

New design (pp. 217–240): a State interface with the four actions; one class per state (NoQuarterState, HasQuarterState, SoldState, SoldOutState, WinnerState); GumballMachine holds the state objects and the current state and delegates every action to it; states call machine.setState(next).

MEMORIZE **State** — Allow an object to alter its behavior when its internal state changes. The object will appear to change its class (p. 241).

```
Context - state: State                          «interface» State + doThis() + doThat()
 + Context(initialState)                                ^                    ^
 + changeState(s) { state = s; }                 ConcreteStateA        ConcreteStateB
 + doThis() { state.doThis(); }                  - context: Context    (may call
context.changeState(new B()))
```

```
// IJ slides p203-240 — State pattern: gumball machine
interface State { void insertQuarter(); void ejectQuarter(); void turnCrank(); void dispense(); }

class GumballMachine {                                          // context
    final State soldOut, noQuarter, hasQuarter, sold;
    private State state;
    int count;
    GumballMachine(int n) {
        soldOut = new SoldOutState(this); noQuarter = new NoQuarterState(this);
        hasQuarter = new HasQuarterState(this); sold = new SoldState(this);
        count = n; state = n > 0 ? noQuarter : soldOut;
    }
    void setState(State s) { state = s; }
    void insertQuarter() { state.insertQuarter(); }
    void ejectQuarter() { state.ejectQuarter(); }
    void turnCrank() { state.turnCrank(); state.dispense(); }
    void releaseBall() { System.out.println("A gumball comes rolling out"); if (count > 0) count--; }
}
class NoQuarterState implements State {
    private final GumballMachine m; NoQuarterState(GumballMachine m) { this.m = m; }
    public void insertQuarter() { System.out.println("You inserted a quarter");
m.setState(m.hasQuarter); }
    public void ejectQuarter() { System.out.println("You haven't inserted a quarter"); }
    public void turnCrank() { System.out.println("You turned, but there's no quarter"); }
    public void dispense() { System.out.println("You need to pay first"); }
}
class HasQuarterState implements State {
    private final GumballMachine m; HasQuarterState(GumballMachine m) { this.m = m; }
    public void insertQuarter() { System.out.println("You can't insert another quarter"); }
    public void ejectQuarter() { System.out.println("Quarter returned"); m.setState(m.noQuarter); }
    public void turnCrank() { System.out.println("You turned..."); m.setState(m.sold); }
    public void dispense() { System.out.println("No gumball dispensed"); }
}
class SoldState implements State {
    private final GumballMachine m; SoldState(GumballMachine m) { this.m = m; }
    public void insertQuarter() { System.out.println("Please wait, we're already giving you a
gumball"); }
    public void ejectQuarter() { System.out.println("Sorry, you already turned the crank"); }
    public void turnCrank() { System.out.println("Turning twice doesn't get you another gumball!"); }
    public void dispense() {
        m.releaseBall();
        if (m.count > 0) m.setState(m.noQuarter);
        else { System.out.println("Oops, out of gumballs!"); m.setState(m.soldOut); }
    }
}
```

```
class SoldOutState implements State {
    private final GumballMachine m; SoldOutState(GumballMachine m) { this.m = m; }
    public void insertQuarter() { System.out.println("Machine is sold out"); }
    public void ejectQuarter() { System.out.println("You haven't inserted a quarter"); }
    public void turnCrank() { System.out.println("No gumballs"); }
    public void dispense() { System.out.println("No gumball dispensed"); }
}
public class GumballDemo {
    public static void main(String[] args) {
        GumballMachine g = new GumballMachine(1);
        g.insertQuarter(); g.turnCrank();
        g.insertQuarter();                              // sold out now
    }
}
```

Document example (pp. 252–253): publish() behaves differently in Draft (→ Moderation, or Published if admin), Moderation (→ Published if approved) and Published (do nothing).

	State	Strategy
Common	both use composition and delegate to a helper object ('twins separated at birth', p. 254)	
Who changes the helper	the context or the **states themselves**, as a result of actions	the **client** chooses/sets it
Knowledge	states may know each other and the context	strategies are independent and unaware of each other
Models	behaviour that depends on the object's current state (life cycle)	interchangeable algorithms for one task
Example	gumball machine, document workflow, TCP connection	sorting algorithms, payment methods, routes

Practice — Lesson 10

(a) Actual — 2024-25 Q8(c).

(a) State vs Strategy. (10)

✓ Solution

(a) The comparison table above with one diagram each (Context → State with transitions between ConcreteStates; Context → Strategy set by the client) and the gumball vs sorting examples.

Lesson 11 — Identifying the Pattern; Chain of Responsibility and Interpreter

Mixed TF questions (textbook for CoR and Interpreter)

1. Chain of Responsibility (textbook)

Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle it: chain the receivers and pass the request along until one handles it. Handler (successor; handleRequest()) with concrete handlers that either handle or forward.

2. Interpreter (textbook)

Given a language, define a representation of its grammar and an interpreter that evaluates sentences: one class per grammar rule (TerminalExpression, NonterminalExpression), all with interpret(context); a sentence is an abstract syntax tree (a Composite).

Practice — Lesson 11

(a) Actual — 2023-24 Q2(b). (b) Actual — 2021-22 Q4(a). (c) Actual — 2012-13 Q1(c).

(a) HelpHandler defines the interface for help requests, keeps a help topic (empty by default) and a reference to its successor; HandleHelp is overridden by subclasses; HasHelp checks for a topic. Widget (abstract) is a HelpHandler; Button and Dialog are Widgets; Button shows its topic or forwards; Dialog's successor is any handler (an Application); Application is a HelpHandler (not a widget) that gives general help. Draw the UML using the appropriate pattern. (8)

(b) Identify the most specific pattern: (i) NuetFlix recommends a newly released movie to users interested in its genre (Horror, Thriller, Comedy) (ii) jit: kommit / pussh / stashh (iii) BanglaPhone switches between a stable-but-slow and a fast-but-unstable implementation depending on the number of users under a cell tower, without affecting other code (iv) R2-D2 journal: authors and reviewers communicate only through an internal team, never knowing each other's identity. (20)

(c) An application takes 'A op1 B op2 C' (numbers, operators + − * /) and returns the result. Which pattern and why? Class diagram. (4+10)

✓ Solution

(a) Chain of Responsibility (the GoF help example).

```
HelpHandler {abstract} - successor: HelpHandler - topic = "" ---(successor 0..1)---> HelpHandler
  + hasHelp(): boolean  + handleHelp() { if (successor != null) successor.handleHelp(); }
     ^ Widget {abstract}                          ^ Application + handleHelp() (general help / topic list)
          ^ Button + handleHelp() { if hasHelp() show topic else super.handleHelp(); }
          ^ Dialog + handleHelp() { same; successor = an Application (any HelpHandler) }
Chain at run time:  Button --> Dialog --> Application
```

```
// 2023-24 Q2(b) — Chain of Responsibility: context-sensitive help (GoF HelpHandler)
abstract class HelpHandler {
    private final HelpHandler successor;
    private final String topic;                          // empty by default
    HelpHandler(HelpHandler successor, String topic) { this.successor = successor; this.topic =
topic; }
    boolean hasHelp() { return topic != null && !topic.isEmpty(); }
    void handleHelp() {
        if (successor != null) successor.handleHelp();
        else System.out.println("No help available");
    }
    String topic() { return topic; }
}
abstract class Widget extends HelpHandler {
    Widget(Widget parent, String topic) { super(parent, topic); }
}
class Button extends Widget {
    Button(Widget parent, String topic) { super(parent, topic); }
    void handleHelp() {
        if (hasHelp()) System.out.println("Button help: " + topic());
        else super.handleHelp();                         // forward to successor
    }
}
class Dialog extends Widget {
    Dialog(HelpHandler h, String topic) { super(null, topic); this.next = h; }
    private final HelpHandler next;                      // successor may be any handler
    void handleHelp() {
        if (hasHelp()) System.out.println("Dialog help: " + topic());
        else if (next != null) next.handleHelp();
    }
}
class Application extends HelpHandler {
    Application(String topic) { super(null, topic); }
    void handleHelp() { System.out.println("Application: general help / list of help topics"); }
}
public class HelpDemo {
    public static void main(String[] args) {
        Application app = new Application("main");
        Dialog dialog = new Dialog(app, "");              // no topic -> forwards to Application
        Button ok = new Button(dialog, "");               // no topic -> forwards to Dialog
        Button print = new Button(dialog, "Prints the document");
        print.handleHelp();
        ok.handleHelp();
    }
}
```

(b)

Case	Pattern	Justification
(i) NuetFlix	Observer	users subscribe to genres (one subject per genre); a release notifies all subscribers of that genre
(ii) jit	Command	each operation is an object with execute() acting on the repository; can be logged/queued/undone (VcsDemo above)
(iii) BanglaPhone	Strategy (State also accepted)	two interchangeable implementations of the same service selected at run time by user count; context code unchanged. If the switch is driven internally by the tower's load as a life-cycle, call it State — justify either
(iv) R2-D2	Mediator	authors and reviewers never reference each other; the committee's internal team routes all submissions, reviews and decisions, hiding identities

```
(i)  Genre (Subject) + subscribe(u), unsubscribe(u), release(movie) -> for u: u.recommend(movie)
     User (Observer) + recommend(movie)          (a user may subscribe to Horror, Thriller and/or Comedy)
(iv) «interface» JournalMediator + submit(author, paper), review(reviewer, paperId, comments), decide(...)
     EditorialTeam (ConcreteMediator) — Author and Reviewer hold only the mediator
```

(c) Interpreter: the input is a sentence of a tiny grammar (expr := number | expr op expr); build an expression tree and evaluate it. Precedence is handled by the parser (* and / bind tighter). (Strategy for

the four operators is an acceptable simpler answer: each operator is an interchangeable Operation strategy.)

```
«interface» Expression + interpret(): double
   ^ NumberExpression - value                                  (terminal)
   ^ BinaryExpression {abstract} - left, right: Expression     (non-terminal, Composite)
        ^ AddExpression ^ SubtractExpression ^ MultiplyExpression ^ DivideExpression
Parser + parse("A op1 B op2 C"): Expression    Client: parser.parse(s).interpret()
```

Quick Notes — Behavioral Patterns

Revision points for Lessons 5–11.

L5 · Strategy

- Context has-a Strategy; setStrategy() at run time. Ducks, robots, sorting, schemes, grading.

L6 · Template Method

- final templateMethod() calls steps; abstract steps + hooks. Inheritance vs Strategy's composition.

L7 · Observer

- Subject register/remove/notify; Observer update(). Push vs pull. Stock, blog, dashboard, news, groups.

L8 · Mediator

- Colleagues call mediator.notify(this, event). ATC, dialog, sort broker. vs Observer: mutual vs one-way.

L9 · Command

- execute(); invoker, receiver; queue, history, undo. Smart-home replay, jit.

L10 · State

- Context delegates to current State; states switch themselves. Gumball. vs Strategy: who changes it.

L11 · Identify

- CoR = pass along the chain (HelpHandler); Interpreter = grammar → tree → interpret().